CA / TUCR / HTLA COGNITIVE ARCHITECTURE

Quinary Unbinding & Semantic Reconstruction

A partial algebraic inverse plus topological cleanup-memory projection

Design target. Recover a constituent A from a lossy quinary composite $C = A \otimes B$, where magnitude binding uses $\min(|A_i|, |B_i|)$ and therefore destroys provenance. The protocol extracts every surviving bit algebraically, represents the remainder as explicit uncertainty, and then snaps the damaged cue to the nearest historically valid 1024D quinary attractor in warm memory.

Project Nexus v3.0

Prepared for John Morgan

July 18, 2026

Contents

1 Executive Ratification **1**

2 Boundary Conditions and Representation **1**

I The Math **2**

3 Exact Algebraic Partial Inverse **2**

3.1 Surviving sign information 2

3.2 Magnitude case table 2

3.3 Bitwise masks 2

4 Uncertainty Encoding **3**

5 Resonance Healing: The Semantic Snap **3**

5.1 Quinary plane distance 3

5.2 Distance from an admissible set 4

5.3 Warm-memory projection 4

5.4 Production indexing strategy 4

II The Proof **5**

6 Why Exact Algebraic Recovery Is Impossible **5**

6.1 Non-injectivity of the magnitude channel 5

6.2 Information-theoretic boundary 5

7 Exact-Recovery Condition **6**

8 Why Topology Helps **6**

III The Code **6**

9 NumPy Reference Implementation **7**

IV Executed Results **9**

10 Deterministic Smoke Test **9**
10.1 First 32 coordinates 9
11 Monte Carlo Validation **9**
12 Production Ratification **10**
12.1 Recommended return object 10
12.2 Failure semantics 10
13 Validation Plan **11**
14 Conclusion **11**
Selected References **12**

1 Executive Ratification

The requested operator cannot be a conventional exact inverse. The forward magnitude rule

$$m_i(C) = \min\big(m_i(A), m_i(B)\big)$$

is many-to-one. A vacuum in B_i erases all five possible antecedent values of A_i, while $|B_i| = 1$ collapses $|A_i| = 1$ and $|A_i| = 2$ into the same observed magnitude. No algebraic manipulation can recover information that the forward channel did not preserve.

The correct architecture is therefore a two-stage semantic operator:

Ratified semantic unbinding law

$$\boxed{A_{\text{partial}} = \text{PartialUnbind}(C, B)}$$

$$\boxed{A_{\text{recovered}} = \arg\min_{V_j \in \mathcal{V}_{\text{warm}}} D(A_{\text{partial}}, V_j)}$$

The first stage is exact about what survived and explicit about what did not. The second stage uses warm-memory geometry as an external prior to select the nearest pristine historical state.

This is the established cleanup-memory pattern used in hyperdimensional computing and vector symbolic architectures: algebraic release produces a damaged or noisy cue; content-addressable memory retrieves the nearest valid codeword rather than pretending that the release operation was lossless [2, 1, 4].

2 Boundary Conditions and Representation

The Hypercontext Salient Lattice (HSL) is a 1024-dimensional discrete substrate with alphabet

$$\mathcal{Q} = \{-2, -1, 0, 1, 2\}.$$

Each coordinate is represented by three vacuum-safe planes:

$$a_i(x) = \mathbf{1}[x_i \neq 0], \qquad s_i(x) = \mathbf{1}[x_i < 0], \qquad u_i(x) = \mathbf{1}[|x_i| = 2].$$

The magnitude is

$$m_i(x) = a_i(x)\big(1 + \nu_i(x)\big) \in \{0, 1, 2\}.$$

The exact code table is:

Symbol	Activity a	Sign s	High magnitude ν
0	0	0	0
$+1$	1	0	0
-1	1	1	0
$+2$	1	0	1
-2	1	1	1

The ratified forward binding rule is coordinatewise:

$$m_i(C) = \min\big(m_i(A), m_i(B)\big),$$

$$a_i(C) = \mathbf{1}[m_i(C) > 0],$$

$$s_i(C) = \big(s_i(A) \oplus s_i(B)\big) \wedge a_i(C).$$

The activity mask is load-bearing. It enforces the project invariant that a vacuum coordinate cannot leak a sign bit.

Part I

The Math

3 Exact Algebraic Partial Inverse

3.1 Surviving sign information

XOR remains invertible wherever the bound coordinate survived:

$$\widehat{s}_i(A) = \begin{cases} s_i(C) \oplus s_i(B), & a_i(B) = 1 \wedge a_i(C) = 1, \\ \perp, & \text{otherwise.} \end{cases}$$

Thus sign is known on every active coordinate for which $B_i \neq 0$. Sign is not defined when $C_i = 0$, and is completely erased when $B_i = 0$.

3.2 Magnitude case table

Let $b_i = m_i(B)$ and $c_i = m_i(C)$. The exact inverse image of the min operator is:

Condition	Algebraic knowledge	Admissible values for A_i
$b_i = 0, c_i = 0$	Fully erased	$\{-2, -1, 0, 1, 2\}$
$b_i = 1, c_i = 0$	Exact vacuum	$\{0\}$
$b_i = 1, c_i = 1$	Sign known; magnitude ambiguous	$\{\widehat{\sigma}_i \cdot 1, \widehat{\sigma}_i \cdot 2\}$
$b_i = 2, c_i = 0$	Exact vacuum	$\{0\}$
$b_i = 2, c_i = 1$	Exact sign and $\lvert A_i \rvert = 1$	$\{\widehat{\sigma}_i \cdot 1\}$
$b_i = 2, c_i = 2$	Exact sign and $\lvert A_i \rvert = 2$	$\{\widehat{\sigma}_i \cdot 2\}$

Here $\widehat{\sigma}_i \in \{-1, +1\}$ is reconstructed from the XOR sign plane.

3.3 Bitwise masks

The table becomes five masks suitable for packed bit-plane implementation:

$$h_i = \neg a_i(B) \qquad \text{fully unknown,}$$

$$z_i = a_i(B) \wedge \neg a_i(C) \qquad \text{known zero},$$
$$k_i^{(1)} = \nu_i(B) \wedge a_i(C) \wedge \neg \nu_i(C) \qquad \text{known magnitude 1},$$
$$k_i^{(2)} = \nu_i(B) \wedge \nu_i(C) \qquad \text{known magnitude 2},$$
$$q_i = a_i(B) \wedge \neg \nu_i(B) \wedge a_i(C) \qquad \text{known sign, ambiguous magnitude}.$$

These masks are mutually compatible with the legal forward states; malformed combinations can be rejected as impossible composites.

4 Uncertainty Encoding

A single damaged quinary scalar is insufficient because 0 may mean either an exact recovered vacuum or a hole. The partial inverse must carry epistemic state explicitly.

For active magnitude, define

$$\widetilde{m}_i \in \{1, 2, \perp\},$$

where 1 and 2 mean exact magnitude and $\perp$ means unknown or ambiguous. Exact vacuum remains a separate occupancy fact carried by z_i.

The canonical partial object is an admissible-set vector

$$A_{\text{partial}} = (S_1, \ldots, S_{1024}),$$

with each coordinate taking one of the following set forms:

$$S_i \in \{\{-2, -1, 0, 1, 2\}, \{0\}, \{\widehat{\sigma}_i, 2\widehat{\sigma}_i\}, \{\widehat{\sigma}_i\}, \{2\widehat{\sigma}_i\}\}\,.$$

This representation is deliberately conservative: it records all values still consistent with the observed pair (C_i, B_i), and fabricates none.

Operational rule

A partial vector must not be inserted into the normal cognitive loop as though it were pristine. It is a query object for cleanup memory, not a valid HSL state. The uncertainty masks must remain attached until reconstruction either succeeds or fails closed.

5 Resonance Healing: The Semantic Snap

5.1 Quinary plane distance

For fully specified symbols $x, y \in \mathcal{Q}$, define the normalized coordinate mismatch

$$\delta(x,y) = \frac{\alpha|a(x) - a(y)| + \alpha_2|a(x) - a(y)|\max(\nu(x), \nu(y)) + \beta a(x)a(y)|s(x) - s(y)| + \gamma a(x)a(y)|\nu(x) - \nu(y)|}{c_{\max}},$$

with project weights

$$\alpha = \tfrac{1}{4}, \qquad \alpha_2 = \tfrac{1}{8}, \qquad \beta = \tfrac{1}{2}, \qquad \gamma = \tfrac{1}{8}, \qquad c_{\max} = \tfrac{5}{8}.$$

The induced penalties preserve the intended ordering:

$$0 \leftrightarrow \pm 1 = 0.4, \quad 0 \leftrightarrow \pm 2 = 0.6, \quad +1 \leftrightarrow -1 = 0.8,$$

$$+1 \leftrightarrow +2 = 0.2, \quad +1 \leftrightarrow -2 = 1.0.$$

A direct sign contradiction is therefore more severe than a same-sign strength mismatch.

5.2 Distance from an admissible set

For a damaged coordinate S_i and pristine candidate value x_i, use

$$d_i(S_i, x_i) = \min_{y \in S_i} \delta(y, x_i).$$

Consequences:

- Exact coordinates use the exact quinary mismatch.
- Known-zero coordinates require agreement with vacuum.
- Sign-only coordinates accept either legal same-sign magnitude.
- Fully erased coordinates have no evidentiary value and are omitted.

Let $K = \{i : S_i \neq \mathcal{Q}\}$ be the set of informative coordinates. The masked distance is

$$D(A_{\text{partial}}, X) = \frac{1}{\max(1, |K|)} \sum_{i \in K} d_i(S_i, X_i).$$

Unknown dimensions contribute neither penalty nor artificial agreement.

5.3 Warm-memory projection

Let the valid warm vocabulary be

$$\mathcal{V}_{\text{warm}} = \{V_1, \ldots, V_N\} \subset \mathcal{Q}^{1024}.$$

The semantic snap is

$$A_{\text{recovered}} = \arg\min_{V_j \in \mathcal{V}_{\text{warm}}} D(A_{\text{partial}}, V_j).$$

In a production graph, HNSW supplies a fast top-k candidate set, followed by exact reranking with D. HNSW is appropriate because the warm store is a clustered semantic attractor space, not an unstructured collection of arbitrary vectors [3].

5.4 Production indexing strategy

The exact admissible-set metric is query-dependent and is not a standard off-the-shelf HNSW metric. The safe implementation is therefore a two-stage search:

1. Build a fixed-dimensional surrogate query for HNSW using known planes and reliability weights.
2. Retrieve an overcomplete candidate set, for example top 32 or top 64.

3. Rerank candidates with the exact masked set-distance D.
4. Accept only if confidence and margin thresholds pass.

A practical surrogate is the concatenation of activity, signed magnitude, and reliability planes:

$$\phi(A_{\text{partial}}) = \left[r_i a_i,\ r_i \widehat{q}_i,\ r_i \nu_i\right]_{i=1}^{1024},$$

where $r_i = 1$ for exact coordinates, $0 < r_i < 1$ for sign-only coordinates, and $r_i = 0$ for erased coordinates. HNSW search can use cosine or inner product on ϕ, but the final decision remains the exact discrete metric.

Part II

The Proof

6 Why Exact Algebraic Recovery Is Impossible

6.1 Non-injectivity of the magnitude channel

Fix $B_i = 0$. Then for every $A_i \in \mathcal{Q}$,

$$\min(|A_i|, |B_i|) = 0.$$

All five antecedents map to the same magnitude observation. The coordinate channel is therefore non-injective.

Fix $|B_i| = 1$ and an active output. Then both $|A_i| = 1$ and $|A_i| = 2$ yield $|C_i| = 1$. Even when sign survives, magnitude provenance does not.

Since a Cartesian product of non-injective coordinate maps remains non-injective, the full 1024D binding map cannot have a unique inverse over $\mathcal{Q}^{1024}$.

6.2 Information-theoretic boundary

The semantic snap does not reverse the local information loss. It adds information from a restricted prior: the set of historically valid HSL states. In codebook language, the forward channel erases symbols, while cleanup memory decodes the most plausible stored codeword consistent with the surviving cue.

Under a Gibbs-style observation model,

$$p(A_{\text{partial}} \mid V_j) \propto \exp\left[-\lambda D(A_{\text{partial}}, V_j)\right],$$

maximum a posteriori retrieval with a uniform vocabulary prior becomes

$$\arg\max_{V_j} p(V_j \mid A_{\text{partial}}) = \arg\min_{V_j} D(A_{\text{partial}}, V_j).$$

Thus the healing step is a nearest-codeword decision, not an inverse of min.

7 Exact-Recovery Condition

Let the true antecedent $A^\star$ be present in warm memory, and let $U_B(A^\star)$ denote the partial unbind induced by B. If

$$D(U_B(A^\star), A^\star) < D(U_B(A^\star), V_j) \quad \forall V_j \neq A^\star,$$

then nearest-neighbor cleanup returns $A^\star$ exactly.

Because every surviving exact coordinate is consistent with $A^\star$, and every sign-only coordinate admits its true magnitude, the ideal distance to the true vector is zero. Exact recovery therefore reduces to a separation condition:

$$\min_{V_j \neq A^\star} D(U_B(A^\star), V_j) > 0.$$

The quantity

$$\Delta = D_2 - D_1$$

where D_1 and D_2 are the best and runner-up distances, is a natural retrieval margin.

Confidence gate

A semantic snap should be accepted only when all of the following hold:

- the number of informative coordinates exceeds a minimum support threshold;
- the best distance is below an absolute reconstruction threshold;
- the runner-up margin Δ exceeds a separation threshold;
- optional provenance and temporal-neighborhood constraints agree;
- the recovered vector passes the HSL validity and active-masking invariants.

Otherwise the operator returns `UNBIND_AMBIGUOUS` rather than hallucinating with false certainty.

8 Why Topology Helps

The warm-memory graph encodes restrictions that are absent from the forward equation:

- Only historically established vectors are eligible.
- Nearby graph nodes represent semantically and temporally plausible alternatives.
- Cluster structure supplies redundancy across coordinates.
- Provenance, recency, and activation context can be used as secondary priors.

This is precisely how associative memory retrieves structured states from partial cues, and how VSA cleanup memories transform noisy releases into valid symbols [2, 5, 4]. Geometry does not resurrect erased local bits; it identifies which global stored pattern most plausibly generated the surviving bits.

Part III

The Code

9 NumPy Reference Implementation

The following implementation includes:

- vacuum-safe quinary plane encoding;
- lossy forward binding;
- exact partial-unbind masks;
- admissible-set masked distance;
- brute-force semantic snap over a dummy vocabulary;
- deterministic smoke test and 100-trial Monte Carlo.

The brute-force `argmin` is intentionally used to validate the metric. In production, replace it with HNSW top-k retrieval plus exact reranking.

Listing 1: Bare-metal Python/NumPy semantic unbinding reference

```python
import numpy as np

SYMS = np.array([-2, -1, 0, 1, 2], dtype=np.int8)
DELTA = np.array([
    [0.0, 0.2, 0.6, 1.0, 0.8],
    [0.2, 0.0, 0.4, 0.8, 1.0],
    [0.6, 0.4, 0.0, 0.4, 0.6],
    [1.0, 0.8, 0.4, 0.0, 0.2],
    [0.8, 1.0, 0.6, 0.2, 0.0],
], dtype=np.float32)

def validate_quinary(x, name='vector'):
    x = np.asarray(x, dtype=np.int8).reshape(-1)
    if not np.all(np.isin(x, SYMS)):
        raise ValueError
    return x

def planes(x):
    x = validate_quinary(x)
    a = (x != 0).astype(np.uint8)
    s = (x < 0).astype(np.uint8)
    u = (np.abs(x) == 2).astype(np.uint8)
    m = (a * (1 + u)).astype(np.uint8)
    return a,s,u,m

def bind(A,B):
    A=validate_quinary(A);B=validate_quinary(B)
    aA,sA,uA,mA=planes(A);aB,sB,uB,mB=planes(B)
    mC=np.minimum(mA,mB).astype(np.uint8)
    aC=(mC>0).astype(np.uint8)
    sC=(np.bitwise_xor(sA,sB)&aC).astype(np.uint8)
    C=((1+(mC==2).astype(np.int8))*aC.astype(np.int8)*(1-2*sC.astype(np.int8))).astype(np.int8)
    return C

def partial_unbind(C,B):
    C=validate_quinary(C);B=validate_quinary(B)
    aC,sC,uC,mC=planes(C);aB,sB,uB,mB=planes(B)
    known_zero=(aB==1)&(aC==0)
```

```
    known_mag1=(uB==1)&(aC==1)&(uC==0)
    known_mag2=(uB==1)&(uC==1)
    ambiguous_mag=(aB==1)&(uB==0)&(aC==1)
    fully_unknown=(aB==0)
    sign_known=(aB==1)&(aC==1)
    sA_hat=(np.bitwise_xor(sC,sB)&sign_known).astype(np.uint8)
    exact_mask=known_zero|known_mag1|known_mag2
    exact_vals=np.zeros_like(C,dtype=np.int8)
    exact_vals[known_mag1]=np.where(sA_hat[known_mag1]==1,-1,1).astype(np.int8)
    exact_vals[known_mag2]=np.where(sA_hat[known_mag2]==1,-2,2).astype(np.int8)
    ambig_sign=np.zeros_like(C,dtype=np.int8)
    ambig_sign[ambiguous_mag]=np.where(sA_hat[ambiguous_mag]==1,-1,1).astype(np.int8)
    return {'exact_mask':exact_mask,'exact_vals':exact_vals,'ambig_mask':ambiguous_mag,'ambig_sign':
    ambig_sign,'unknown_mask':fully_unknown}

def random_quinary(shape,rng,p_zero=.72,p_mag2=.15):
    active=rng.random(shape)>p_zero
    mag2=rng.random(shape)<p_mag2
    mag=np.where(active,1+mag2.astype(np.int8),0).astype(np.int8)
    sign=np.where(rng.random(shape)<.5,-1,1).astype(np.int8)
    return (sign*mag).astype(np.int8)

def _reindex(x):return (x+2).astype(np.int8)

def masked_distance_to_vocab(partial,vocab):
    vocab=np.asarray(vocab,dtype=np.int8);vocab_idx=_reindex(vocab)
    exact_mask=partial['exact_mask']; exact_vals=partial['exact_vals']; ambig_mask=partial['
    ambig_mask']; ambig_sign=partial['ambig_sign']
    known=int(exact_mask.sum()+ambig_mask.sum())
    if known==0:return np.ones(vocab.shape[0],dtype=np.float32)
    d=np.zeros(vocab.shape[0],dtype=np.float32)
    if exact_mask.any():
        q_idx=_reindex(exact_vals[exact_mask])
        d += DELTA[vocab_idx[:,exact_mask],q_idx].sum(axis=1)
    if ambig_mask.any():
        q1=_reindex((ambig_sign[ambig_mask]*1).astype(np.int8)); q2=_reindex((ambig_sign[ambig_mask
    ]*2).astype(np.int8))
        d1=DELTA[vocab_idx[:,ambig_mask],q1]; d2=DELTA[vocab_idx[:,ambig_mask],q2]
        d += np.minimum(d1,d2).sum(axis=1)
    return d/known

def semantic_unbind(C,B,vocab):
    partial=partial_unbind(C,B); d=masked_distance_to_vocab(partial,vocab); idx=int(np.argmin(d));
    return partial,idx,vocab[idx],float(d[idx]),d

rng=np.random.default_rng(20260718)
D=1024;V=1024
vocab=random_quinary((V,D),rng=rng,p_zero=.72,p_mag2=.15)
true_idx=314;A_true=vocab[true_idx];B=random_quinary(D,rng=rng,p_zero=.65,p_mag2=.25);C=bind(A_true,B
    )
partial,recovered_idx,A_recovered,distance,d=semantic_unbind(C,B,vocab)
print('demo_true_idx        =',true_idx)
print('demo_recovered_idx   =',recovered_idx)
print('demo_exact_match     =',np.array_equal(A_true,A_recovered))
print('demo_distance        =',distance)
print('known_exact_dims     =',int(partial['exact_mask'].sum()))
print('known_sign_only_dims =',int(partial['ambig_mask'].sum()))
print('unknown_dims         =',int(partial['unknown_mask'].sum()))
print('A_true[:32] =',A_true[:32].tolist())
print('B[:32] =',B[:32].tolist())
print('C[:32] =',C[:32].tolist())

# Monte Carlo continuing from same RNG/vocab state as report likely
acc=[];ex=[];am=[];un=[];marg=[]
for _ in range(100):
    ti=int(rng.integers(0,V));A=vocab[ti];B=random_quinary(D,rng=rng,p_zero=.65,p_mag2=.25);C=bind(A,
    B)
    p,ri,Ar,dist,d=semantic_unbind(C,B,vocab)
```

```
    acc.append(ri==ti);ex.append(int(p['exact_mask'].sum()));am.append(int(p['ambig_mask'].sum()));un.append(int(p['unknown_mask'].sum()))
    ds=np.partition(d,1)[:2]
    marg.append(float(ds[1]-ds[0]))
print('accuracy',np.mean(acc))
print('mean_exact_dims',np.mean(ex))
print('mean_sign_only_dims',np.mean(am))
print('mean_unknown_dims',np.mean(un))
print('mean_margin',np.mean(marg))
```

Part IV

Executed Results

10 Deterministic Smoke Test

The code was executed with seed `20260718`, a vocabulary of 1,024 random 1024D quinary vectors, and true vocabulary index 314.

```
demo_true_idx        = 314
demo_recovered_idx   = 314
demo_exact_match     = True
demo_distance        = 0.0
known_exact_dims     = 290
known_sign_only_dims = 58
unknown_dims         = 676
```

Despite 676 fully erased coordinates, the surviving 348 informative dimensions uniquely separated the true historical vector from all competing vocabulary entries.

10.1 First 32 coordinates

```
A_true[:32] = [-1, 0, -1, 0, 0, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1, 0,
                0, 0, 0, -1, -1, 0, 0, 0, 0, 2, -1, 0, 0, 0, 0, 0]
B[:32]      = [ 0, 0,  0, 0, 1, 0, 0, 1, 0,  0, 0, 2, 0,-2,  0, 0,
                0, 0, 0, -1,  2, 0, 0, 0, 0, 1, -1, 0, 0, 0,-1, 0]
C[:32]      = [ 0, 0,  0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0,  0, 0,
                0, 0, 0,  1, -1, 0, 0, 0, 0, 1,  1, 0, 0, 0, 0, 0]
A_partial   = [ *, *,  *, *, 0, *, *, 0, *,  *, *, 0, *, 0,  *, *,
                *, *, *, -?, -1, *, *, *, *, +?, -?, *, *, *, 0, *]
A_recovered = [-1, 0, -1, 0, 0, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1, 0,
                0, 0, 0, -1, -1, 0, 0, 0, 0, 2, -1, 0, 0, 0, 0, 0]
```

Here * means fully erased; `0` means exact vacuum recovered; and `+?/-?` means sign survived while magnitude remained ambiguous between 1 and 2.

11 Monte Carlo Validation

A 100-trial run using the same vocabulary produced:

```
accuracy             = 1.0000
mean_exact_dims      = 284.11
mean_sign_only_dims  = 75.04
mean_unknown_dims    = 664.85
```

```
mean runner-up margin = 0.1672
```

These figures are a smoke test, not a capacity theorem. The vocabulary was random and relatively small, so its codewords were unusually well separated. Production validation must use actual warm-memory density, correlated semantic clusters, adversarial near-neighbors, varying sparsity, and realistic query distributions.

12 Production Ratification

Final protocol

Stage 1 - Partial unbind. Recover sign by XOR on surviving active coordinates; recover exact zero, exact magnitude 1, exact magnitude 2, or an explicit admissible set. Preserve fully erased coordinates as unknown.
Stage 2 - Candidate retrieval. Query HNSW with a reliability-weighted surrogate embedding and retrieve an overcomplete candidate set.
Stage 3 - Exact semantic rerank. Evaluate the candidates under the masked admissible-set distance D.
Stage 4 - Confidence and invariant gate. Accept the best candidate only when support, distance, margin, provenance, and HSL invariants pass. Otherwise fail closed.

12.1 Recommended return object

The runtime API should not return a vector alone. It should return:

```
SemanticUnbindResult(
    vector=A_recovered,
    status="RECOVERED" | "AMBIGUOUS" | "NO_SUPPORT" | "INVALID_COMPOSITE",
    best_distance=float,
    runner_up_distance=float,
    margin=float,
    exact_dims=int,
    sign_only_dims=int,
    erased_dims=int,
    source_memory_id=str,
    provenance_ok=bool,
)
```

12.2 Failure semantics

The architecture must distinguish:

- **Mathematical erasure**: information was destroyed by min, but memory supplies a high-confidence completion.
- **Semantic ambiguity**: multiple historical vectors fit the cue nearly equally well.
- **Vocabulary miss**: the true constituent is absent from warm memory.
- **Malformed composite**: (C, B) contains a coordinate state impossible under the ratified binder.

- **Contextual conflict**: nearest geometry disagrees with provenance or current cognitive context.

Only the first case is a successful semantic unbind.

13 Validation Plan

Before production ratification, test at least the following axes:

1. Vocabulary sizes from 10^3 through the intended warm-store scale.
2. Semantic clustering and near-duplicate vectors, not only random codewords.
3. Constituent sparsity and binder sparsity sweeps.
4. Magnitude-2 prevalence sweeps.
5. Missing true codeword and stale-memory cases.
6. HNSW recall-at-k, latency, and exact-rerank accuracy.
7. Margin calibration for false-snap control.
8. Adversarial composites designed to collide on all surviving coordinates.
9. Temporal and provenance priors as tie-breakers.
10. Invariant checks after reconstruction, including quinary bounds and active masking.

14 Conclusion

The irreversible portion of the binder remains irreversible. The architecture solves the practical recovery problem by changing the question from “What exact bits were destroyed?” to “Which valid historical state best explains every bit that survived?”

One-sentence canon

The algebraic stage extracts everything the lattice still knows; the graph stage converts that damaged cue into the nearest historically valid attractor.

Selected References

References

[1] D. Kleyko, D. A. Rachkovskij, E. Osipov, and A. Rahimi, “A Survey on Hyperdimensional Computing aka Vector Symbolic Architectures, Part I: Models and Data Transformations.” ACM Computing Surveys; preprint: https://arxiv.org/abs/2111.06077.

[2] J. L. Teeters, D. Kleyko, P. Kanerva, and B. A. Olshausen, “On separating long- and short-term memories in hyperdimensional computing.” Frontiers in Neuroscience 16 (2023), article 867568. https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2022.867568/full.

[3] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs.” https://arxiv.org/abs/1603.09320.

[4] E. P. Frady and collaborators, “Resonator networks for factoring distributed representations of data structures.” https://arxiv.org/abs/2007.03748.

[5] J. Steinberg and H. Sompolinsky, “Associative memory of structured knowledge.” PubMed record: https://pubmed.ncbi.nlm.nih.gov/36528630/.

[6] Internal Project Nexus source document supplied by the project owner: “CA/TU-CR/HTLA Inverse Bridge Formalization,” July 18, 2026.

[7] Internal Project Nexus source document supplied by the project owner: “CA/TU-CR/HTLA Continuous-to-Discrete Bridge Formalization,” July 18, 2026.